

PROJECT REPORT

DIGITAL CONTACT BOOK

CSE1021 – Introduction to Problem Solving and Programming

Project Type	Build Your Own Project – VITyarthi
Programming Language	Python
Application Type	Command-Line Contact Management System
Primary Concepts	Lists, Dictionaries, Functions, Conditions, Loops
Storage	In-memory Python data structure

Submitted as an academic project for the CSE1021 course.

Prepared using the project requirements and course syllabus supplied for this project.

1. Introduction

Digital Contact Book is a simple command-line application developed in Python for storing and managing contact information. The system allows a user to add, view, search, update, and delete contacts through a menu-driven interface. Each contact contains a name, phone number, and email address.

The project is designed around the concepts taught in CSE1021 – Introduction to Problem Solving and Programming. The syllabus emphasizes problem solving, algorithms, Python data and expressions, functions, conditional statements, iteration, lists, dictionaries, and practical programming. The project applies these concepts in a small real-world context.

The VITyarthi Build Your Own Project guidelines require a meaningful problem, a technical solution, documentation, functional modules, non-functional requirements, design artefacts, testing, and a project report. This report organizes the Digital Contact Book according to those requirements.

2. Problem Statement

Managing a small set of personal contacts manually can make it difficult to add new information, find a particular contact, modify outdated information, or remove an unwanted entry. The problem addressed by this project is to create a simple program that provides these common contact-management operations through an easy-to-use menu.

Problem: Design and implement a basic computer program that can store contact details and provide efficient operations for adding, displaying, searching, updating, and deleting contacts.

3. Objectives

- Apply basic problem-solving and algorithmic thinking to a real-world task.
- Implement a menu-driven application using Python.
- Use lists and dictionaries to represent and manage contact data.
- Use functions to divide the program into logical operations.
- Apply conditional statements and loops to control program flow.
- Provide basic input validation and duplicate checking.
- Demonstrate practical understanding of concepts covered in CSE1021.

4. Functional Requirements

The VITyarthi guidelines require at least three major functional modules and a clear input/output workflow. The Digital Contact Book provides six menu operations.

Module	Description	Input	Output
Add Contact	Creates a new contact and checks required fields and duplicate phone/email	Name, phone number, email	Success or validation message
View Contacts	Displays all contacts stored in the contact list.	None	List of contacts
Search Contact	Searches contact names using partial, case-insensitive matching	Partial search text	Matching contact details
Update Contact	Finds a contact by phone number and updates selected fields and new values	Phone number, new values	Updated contact/message
Delete Contact	Finds a contact by phone number and removes it.	Phone number	Deletion confirmation/message

Module	Description	Input	Output
Exit	Terminates the menu loop.	Choice 6	Exit message

5. Non-Functional Requirements

Requirement	Project Interpretation
Usability	The program uses a simple numbered menu and clear prompts so a beginner can operate it.
Reliability	The program checks required name/phone input, duplicate phone numbers, and missing contacts before adding.
Maintainability	Operations are separated into functions such as <code>add_contact()</code> , <code>view_contacts()</code> , <code>search_contact()</code> , <code>update_contact()</code> , and <code>delete_contact()</code> .
Resource Efficiency	The application uses basic in-memory Python structures and does not require external libraries or services.
Error Handling	Invalid menu choices and unavailable contacts result in clear messages rather than silent failure.

6. System Architecture

The current implementation follows a simple procedural, menu-driven architecture. The `main()` function acts as the controller. It displays the menu, accepts the user's choice, and calls the appropriate function. Contact data is maintained in the global contacts list. Each contact is represented as a dictionary containing name, phone, and email.

Architecture Flow

User	→	main() Menu	→	Selected Operation
				↓
		contacts list		Add / View / Search / Update / Delete

7. Workflow

The user starts the program and is repeatedly shown the main menu. The selected option determines the operation. After an operation is completed, control returns to the main menu. The loop ends only when the user selects Exit.

Workflow Steps

1. Start the application.
2. Display the Digital Contact Book menu.
3. Read the user's choice.
4. Call the corresponding contact-management function.
5. Validate input and perform the requested operation.
6. Display the result or an appropriate error message.
7. Return to the main menu.
8. Exit when the user selects option 6.

8. Use Case Description

The system has one primary actor: the user. The user interacts with the following use cases:

Actor	Use Case	Purpose
User	Add Contact	Enter and store a new contact.
User	View Contacts	Display all stored contacts.
User	Search Contact	Find contacts by name.
User	Update Contact	Modify an existing contact.
User	Delete Contact	Remove an existing contact.
User	Exit	Terminate the application.

9. Data / Storage Design

The current version does not use a database or permanent file. Contacts are stored in memory using a Python list of dictionaries. This keeps the implementation aligned with the basic Python concepts covered in the course.

Logical Data Structure

contacts → [contact1, contact2, ...]
Each contact → { name, phone, email }

A phone number is treated as the unique identifier for duplicate checking and for locating a contact during update and delete operations.

10. Implementation Details

10.1 Main Program

The main() function contains a while loop that continuously displays the menu and routes the user's selection to the relevant function. The loop terminates when choice == '6'.

10.2 Add Contact

The add_contact() function collects name, phone, and email. It rejects an empty name or phone number and checks every existing contact for a matching phone number before appending the new dictionary to the contacts list.

10.3 Search

The search_contact() function converts both the search text and stored names to lowercase. It then checks whether the search text occurs inside each contact name, allowing partial and case-insensitive searching.

10.4 Update and Delete

Both operations locate a contact by its phone number. Update changes only the fields for which the user provides a new value. Delete removes the matching dictionary from the contacts list.

11. Algorithms and Problem-Solving Approach

The project follows a straightforward top-down problem-solving approach. The overall problem is divided into smaller operations: add, view, search, update, delete, and exit. Each operation is implemented as a separate function.

Operation	Basic Algorithm
Add	Read input → validate required fields → scan for duplicate phone → create dictionary → append to list.
View	Check whether list is empty → iterate through contacts → display each contact.
Search	Read search text → convert to lowercase → scan names → display matching contacts.
Update	Read phone → scan contacts → when matched, read new values → update non-blank fields.
Delete	Read phone → scan contacts → when matched, remove contact → display confirmation.

12. Testing Approach

Testing is performed using functional test cases that exercise the main menu operations and important validation conditions. The following test cases are based on the implemented program behavior.

Test ID	Test Case	Expected Result
TC01	Add valid name, phone, and email.	Contact is added successfully.
TC02	Leave name blank.	Program displays that name and phone are required.
TC03	Leave phone blank.	Program displays that name and phone are required.
TC04	Add a phone number that already exists.	Program rejects the duplicate phone number.
TC05	View contacts when the list is empty.	Program displays 'No contacts available.'
TC06	Search for an existing name.	Matching contact details are displayed.
TC07	Search for a name that does not exist.	Program displays 'Contact not found.'
TC08	Update an existing contact.	Provided non-blank fields are updated.
TC09	Update a missing contact.	Program displays 'Contact not found.'
TC10	Delete an existing contact.	Contact is removed successfully.
TC11	Delete a missing contact.	Program displays 'Contact not found.'
TC12	Enter an invalid menu option.	Program displays an invalid-choice message.

13. Design Decisions and Rationale

Lists and dictionaries: The syllabus specifically includes Python lists and dictionaries. Using a list of dictionaries makes the contact structure easy to understand and directly demonstrates those concepts.

Functions: Each major operation is placed in a separate function. This improves readability and follows the course emphasis on functions.

Command-line interface: A terminal interface avoids unnecessary frameworks and keeps the focus on programming and problem-solving fundamentals.

Phone-based identification: Phone numbers are used to find contacts during update/delete operations and are checked for duplicates during insertion.

In-memory storage: The current version intentionally avoids databases and external packages, keeping the project simple and aligned with the course level.

14. Challenges Faced

- Designing a menu that continues until the user explicitly exits.
- Preventing duplicate phone numbers during contact creation.
- Searching names in a case-insensitive way.
- Allowing partial name searches rather than requiring an exact match.
- Updating only the fields that the user wants to change.
- Handling attempts to update or delete contacts that do not exist.

15. Learnings and Key Takeaways

The project provides practical experience in converting a real-world requirement into a sequence of programming operations. It reinforces the use of variables, strings, lists, dictionaries, functions, conditionals, loops, input/output, validation, and basic search techniques. It also demonstrates how a larger problem can be decomposed into smaller reusable functions.

16. Future Enhancements

- Permanent storage using a text file, CSV file, or database.
- Phone-number and email-format validation.
- Sorting contacts alphabetically.
- Searching by phone number or email.
- Contact groups or categories.
- A graphical user interface.
- Import and export of contact data.

17. Conclusion

Digital Contact Book is a small but practical Python application that applies the core programming concepts taught in CSE1021. The project demonstrates how problem-solving techniques, functions, control flow, lists, and dictionaries can be combined to create a useful application. The menu-driven design keeps the program accessible while providing the essential CRUD-style operations needed for contact management.

The current implementation is intentionally focused on fundamental programming rather than advanced frameworks or databases. This makes it suitable for demonstrating understanding of the course concepts while leaving clear opportunities for future expansion.

18. References

1. CSE1021 – Introduction to Problem Solving and Programming, course syllabus supplied for this project.
2. VITyarthi – Build Your Own Project: General Project Instructions & Submission Guidelines, supplied for this project.
3. Python 3 programming concepts used in the project implementation.